Instituto Tecnológico de Costa Rica, Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación, II Semestre 2026

Curso: IC-6600 Principios de Sistemas Operativos
Profesor: Esteban Arias Méndez
Actividad: Tarea 3, Box de Mr. Meeseeks

Estudiante: Sergio Andrés Zúñiga Castillo
Carné: 2024146884
Fecha de entrega: Octubre, 2026

Box de Mr. Meeseeks: procesos, pipes y semáforos en C

Abstract

This assignment implements, in C on Linux, a single program that simulates the Meeseeks Box from Rick and Morty. The Box reads each request from a menu and forks an evaluator Mr. Meeseeks, which forks the delegates that work on it. Requests travel as private messages through pipes, results and STOP orders travel back the same way, and the level and instance counters live in mmap shared memory protected by a POSIX semaphore. Three kinds of requests are supported: simulated text tasks, whose difficulty from 0 to 100 decides how many helpers are summoned and is diluted at every level, arithmetic and logical operations split among two to four cooperating Meeseeks, and external programs run with execvp() and judged by their exit status. Impossible requests are contained by not summoning helpers from level 6 on, and a configurable timeout declares a planetary chaos that kills the whole process group. Measuring the real time of each request, without the simulated waits, shows that every Meeseeks costs about 130 microseconds, while the total time depends on the simulated attempts and on how many helpers work in parallel.

Índice

1. Introducción....1

2. Diseño e implementación....2

3. Corridas....6

4. Experimentos y análisis....14

5. Conclusiones....16

Referencias....17

1. Introducción

En la serie Rick and Morty, la Meeseeks Box crea a un Mr. Meeseeks cada vez que se presiona su botón: el Meeseeks atiende una solicitud, la cumple y desaparece, y si no puede llama a otros Meeseeks para que le ayuden. La tarea pide modelar esa idea con procesos de Linux: un único programa en C donde la Box es el proceso principal, cada Meeseeks es un proceso creado con fork(), la solicitud viaja por pipes y los contadores compartidos de nivel e instancia se protegen con semáforos [1], [2].

El programa atiende tres casos. En el caso A las solicitudes son textuales y su realización se simula con una dificultad y un tiempo de espera; en el caso B son operaciones aritméticas o lógicas que se calculan; en el caso C se ejecuta un programa externo y se toma en cuenta su código de salida. Para todos se mide el tiempo total y el tiempo real (sin las esperas simuladas), y al despedirse la Box imprime un

informe con cada solicitud. Este documento explica el diseño y el código, muestra corridas de cada caso y analiza los tiempos con cuatro experimentos.

2. Diseño e implementación

2.1 Organización de los procesos

La Box (main) muestra el menú y, por cada solicitud, crea con fork() un Meeseeks evaluador con N = 1 e i = 1. El evaluador es el que decide cómo atender la solicitud según la opción elegida: crea a sus delegados (N = 2, i = 1, 2, ...), les manda la tarea por un pipe propio de cada uno y espera los resultados. En el caso A un delegado que no logra la tarea crea a su vez ayudantes en el nivel siguiente, de modo que se forma un árbol de procesos. Cuando alguien la logra, el Resultado sube por los pipes hasta el evaluador y cada padre manda STOP a los hermanos que siguen trabajando; todos se despiden y terminan en cascada hacia atrás. El evaluador informa quién resolvió la solicitud (pid, ppid, N, i) y le devuelve el control a la Box, que espera con waitpid() y anota los tiempos.

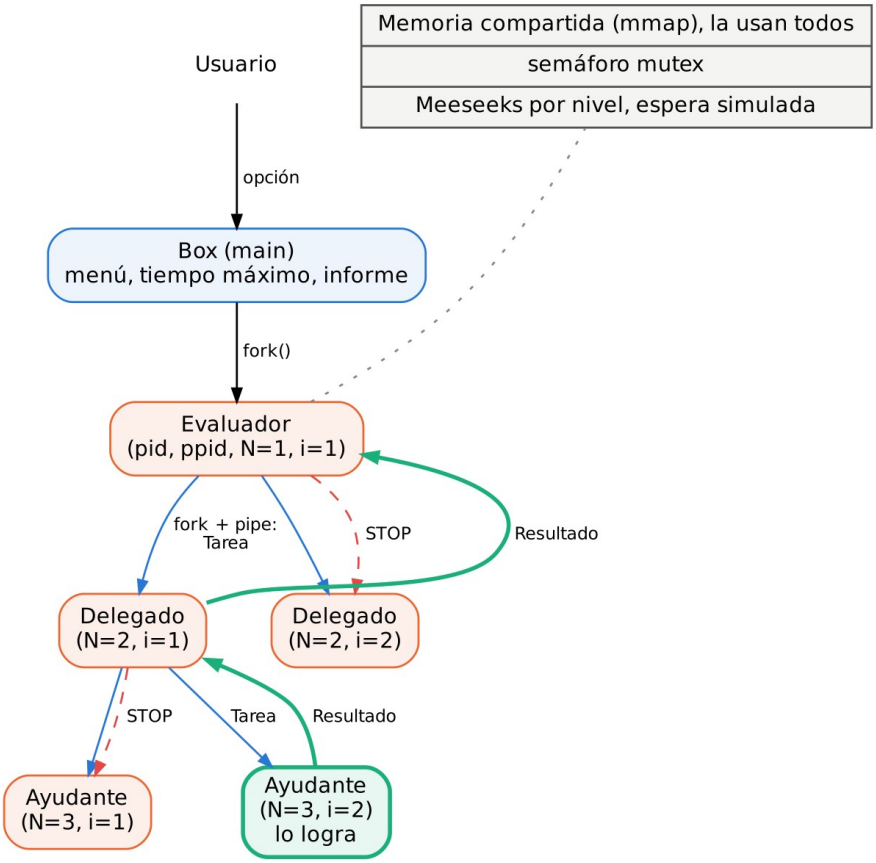


Figura 1. Procesos de una solicitud: la tarea baja por pipes, el Resultado sube y STOP detiene a los hermanos.

Cada opción del menú es una fila de una tabla de opciones (struct Opcion) con su id, nombre, dificultad, los datos que hay que pedirle al usuario y un puntero a la función que ejecuta el evaluador. Los menús son General (caso A), Aritmética y Lógica (caso B), Programas (caso C) y Configuración (tiempo máximo, duración de cada intento y si al vencer el tiempo se decreta el caos o solo se avisa). Todo Meeseeks saluda al nacer con `Hi, I'm Mr. Meeseeks! Look at Meeeee. (pid, ppid, N,`

`i)` y todos sus mensajes salen con la función `decir()`, que antepone su pid y ppid y sangra la línea según el nivel.

2.2 Comunicación por pipes

Ningún mensaje viaja por variables globales. Cada padre crea dos pipes por hijo: uno de bajada, por el que envía la estructura Tarea y, si hace falta, un entero STOP, y uno de subida, por el que el hijo devuelve un Resultado (o un número, en los casos B y C). Para atender a varios hijos a la vez y también a su propio padre, `supervisar()` usa poll() sobre todos esos descriptores [3], [4]. Un delegado del caso A espera su intento con poll() sobre el pipe de su padre, así que un STOP lo interrumpe de inmediato.

```
/* Lo que viaja por el pipe del evaluador a sus delegados */
typedef struct {
    int solicitud;      /* numero de solicitud (#k)                    */
    char texto[64];     /* lo que se pidio                             */
    int dificultad;     /* 0..100, 100 = muy facil; -1 = a la suerte   */
    int duracion_ms;    /* duracion de cada intento; 0 = al azar       */
    int a, b;           /* operandos (aritmetica y logica)             */
    char operacion;     /* operacion logica: '&', '|', '^', '!'        */
    char comando[128];  /* programa por ejecutar                       */
} Tarea;

/* Lo que sube por el pipe de un hijo a su padre */
typedef struct {
    int resuelto;        /* 1 = alguien lo logro                          */
    int pid, ppid, N, i; /* quien lo logro                                */
    double espera;       /* s simulados en la ruta del evaluador al que lo logro */
} Resultado;
```

2.3 Contadores N e i con semáforo

La cantidad de Meeseeks por nivel y la espera simulada de la ruta ganadora están en una estructura Contadores ubicada en memoria compartida (mmap con MAP_SHARED | MAP_ANONYMOUS), creada por la Box antes del primer fork() para que todos los procesos la hereden. Un semáforo POSIX sin nombre, inicializado con pshared = 1, sirve de mutex. N se calcula como el nivel del padre más uno y se registra dentro de la sección crítica; i es el número de hijo que le asigna su padre (1 a k), por lo que es local a cada padre como pide el enunciado.

```
/* Registra un Meeseeks nuevo: su N es el nivel del padre + 1.
 * Seccion critica con el semaforo. Devuelve N. */
int registrar_meeseeks(int nivel_padre)
{
    int N = nivel_padre + 1;
    if (N > MAX_NIVELES)
        N = MAX_NIVELES;
    sem_wait(&cont->mutex);
    cont->instancias[N]++;
    if (N > cont->nivel_max)
        cont->nivel_max = N;
    sem_post(&cont->mutex);
    return N;
}
```

2.4 Caso A: dificultad, ayudantes y plan de contingencia

La dificultad d va de 0 (imposible) a 100 (muy fácil). Se toma de la opción, la escribe el usuario en la solicitud libre o, si responde r, la sortea el evaluador de forma uniforme entre 0 y 100 (caso 2 del enunciado). Cada intento dura lo que se configuró o un tiempo al azar entre 0.5 y 5 s. Al terminar el intento, el Meeseeks lo logra siempre si d > 85 y, si no, con probabilidad d/100. La cantidad de ayudantes

sigue los rangos del enunciado: 1 si d > 85, 2 si 45 < d ≤ 85 y 5 si d ≤ 45. Al repartir la tarea entre k ayudantes, a cada uno le toca una tarea más fácil, de forma parecida al añejamiento (aging): la dificultad se diluye con d + (100 − d) / k^2. Así, con cada nivel la probabilidad sube hasta que la tarea se puede realizar, salvo si d = 0, que sigue siendo imposible.

```
/* Ayudantes segun las instrucciones */
int ayudantes_necesarios(int d)
{
    if (d > 85) return 1;
    if (d > 45) return 2;
    return 5;
}

/* Al repartir la tarea entre k ayudantes, a cada uno le toca mas facil */
int diluir(int d, int k)
{
    if (d == 0)
        return 0;                  /* lo imposible sigue siendo imposible */
    return d + (100 - d) / (k * k);
}
```

El intento de un delegado, y lo que hace si falla, es el centro del caso A:

```
    ...
    if (t.dificultad > 85 || rand() % 100 < t.dificultad) {
        r.resuelto = 1;
        r.pid = getpid();
        r.ppid = getppid();
        r.N = mi_nivel;
        r.i = mi_instancia;
        r.espera = mi_espera;
        decir("Listo! Solicitud #%d completada. Gracias por dejarme servirle. Adios!",
t.solicitud);
        enviar_resultado(fd[1], r);
        exit(0);
    }

    if (mi_nivel >= NIVEL_LIMITE) {
        decir("No pude, y en el nivel %d ya no puedo pedir ayuda. Adios!", mi_nivel);
        enviar_resultado(fd[1], r);
        exit(0);
    }

    Hijo h[MAX_HIJOS];
    int k = ayudantes_necesarios(t.dificultad);
    t.dificultad = diluir(t.dificultad, k);
    decir("No pude! Invoco %d ayudantes (dificultad %d)", k, t.dificultad);
    crear_delegados(t, k, h);
    ...
```

Tabla 1. Delegados y dilución de la dificultad para algunas opciones.

d de la opción	Delegados	d de cada uno	Ayudantes	d siguiente	P(resolver en la 1.ª ronda)
100	1	100	1	100	100 %
80	2	85	2	88	98 %
60	2	70	2	77	91 %
40	5	42	5	44	93 %
20	5	23	5	26	73 %
1	5	4	5	7	18 %

0	5	0	5	0	0 %

El plan de contingencia para las tareas imposibles es un límite de nivel: un Meeseeks con N ≥ 6 que falla ya no pide ayuda y responde que no pudo. Con d = 0 el árbol crece hasta 1 + 5 + 25 + 125 + 625 + 3125 = 3906 Meeseeks y la solicitud termina como no completada, sin colapsar el sistema. Con d = 1 la tarea sí puede lograrse, porque la dilución la vuelve más fácil en cada nivel. Además, como cada ruta tiene a lo sumo 5 intentos (niveles 2 a 6) de 5 s como máximo, ninguna solicitud del caso A puede durar más de unos 25 s.

2.5 Caso B: aritmética y lógica

Las operaciones aritméticas reciben dos enteros y reparten el trabajo entre Meeseeks que se comunican por pipes, incluso entre hermanos:

- **Suma:** un Meeseeks toma ambos números y los suma.
- **Resta:** uno calcula la distancia |a − b| y otro el signo; el evaluador los combina.
- **Multiplicación:** el contador conserva el multiplicador y le resta 1 en cada iteración, avisándole al acumulador, que suma o resta el multiplicando según el signo.
- **División:** el divisor y el dividendo cuentan cuántas veces cabe uno en el otro por restas sucesivas y el residuo calcula lo que sobra. La división entre 0 se reporta como error y la solicitud no se completa.

Las operaciones lógicas (Y, O, XOR y NO, con valores 0 o 1) tienen dificultad 90, así que las resuelve un solo delegado después de un intento simulado, que se resta para obtener el tiempo real.

2.6 Caso C: programas externos

El delegado ejecutor separa el comando en argumentos, crea un proceso nuevo y lo reemplaza con execvp(). Espera con waitpid() y le envía a su padre el código de salida; si exec falla, el hijo termina con 127. El evaluador considera la solicitud completada solo si el código es 0. El menú trae date, ls -l, false (que siempre falla) y la opción de escribir cualquier comando.

```
...
pid_t pid = fork();
if (pid == 0) {
    int nulo = open("/dev/null", O_RDONLY);   /* el programa no lee de la terminal */
    dup2(nulo, STDIN_FILENO);
    signal(SIGPIPE, SIG_DFL);
    execvp(argv[0], argv);
    perror(argv[0]);                          /* solo llega aqui si exec fallo */
    _exit(127);
}
int status, codigo;
waitpid(pid, &status, 0);
codigo = WIFEXITED(status) ? WEXITSTATUS(status) : 128 + WTERMSIG(status);
decir("El programa termino con codigo %d en %.3f s", codigo, ahora() - t0);
enviar_int(fd[1], codigo);
...
```

2.7 Tiempo máximo y caos planetario

El tiempo máximo por solicitud es configurable y empieza en 300 s (5 minutos). Antes de esperar al evaluador, la Box programa alarm(); el manejador de SIGALRM se instala sin SA_RESTART para que waitpid() se interrumpa. Como el evaluador llama a setpgid(0, 0) al nacer, todos los Meeseeks de la

solicitud quedan en su grupo de procesos, y la Box puede terminarlos de una vez con kill(–pgid, SIGKILL), imprimir el informe y cerrar. En la configuración también se puede pedir que solo avise y siga esperando, para medir cuánto tiempo adicional se necesita.

```
    ...
    while (waitpid(pid, &status, 0) < 0) {
        if (errno != EINTR || !tiempo_agotado)
            continue;
        tiempo_agotado = 0;
        if (caos_termina) {
            printf("\n*** CAOS PLANETARIO *** Se superaron los %d s. La Box termina a todos
los Meeseeks.\n",
                   tiempo_maximo);
            kill(-pid, SIGKILL);                  /* todo el grupo del evaluador */
            waitpid(pid, &status, 0);
            sem_init(&cont->mutex, 1, 1);         /* alguien pudo morir con el semaforo
tomado */
            *caos = 1;
            break;
        }
        printf("\n[Box] Se superaron los %d s; sigo esperando a los Meeseeks...\n",
tiempo_maximo);
    ...
```

2.8 Tiempo total y tiempo real

La Box mide el tiempo total con clock_gettime(CLOCK_MONOTONIC) desde el fork() del evaluador hasta que termina. Para el tiempo real, cada delegado mide cuánto esperó en su intento, y al subir el Resultado cada padre le suma su propia espera; así el evaluador conoce la espera simulada de la ruta que resolvió la solicitud y la anota en la memoria compartida. El tiempo real es el total menos esa espera, es decir, lo que costó crear los procesos, comunicarlos y calcular. Si no hubo ruta ganadora o hubo caos, el informe muestra un guion.

3. Corridas

Las corridas se hicieron en Ubuntu 22.04 (kernel 6.8, Intel Core i5-10400F, 2 núcleos asignados), compilando con gcc -Wall -pthread -o meseeks meseeks.c. En cada figura se omiten los menús de la Box y se conservan todos los mensajes de los Meeseeks y el informe final; en los casos con intentos simulados se configuró una duración fija de 1 s para que la salida sea fácil de seguir. Las salidas completas están en informe/corridas.

3.1 Caso A: solicitudes textuales

En la primera corrida se pide Hacer el sandwich perfecto (d = 80). El evaluador delega en 2 Meeseeks con d = 85; los dos lo logran en su primer intento, el primero en responder queda como el que resolvió la solicitud y el informe muestra 3 Meeseeks, 1 s de tiempo total y menos de 1 ms de tiempo real.

```
$ ./meseeks
[pid 10 | ppid 9] Hi, I'm Mr. Meeseeks! Look at Meeeee. (10, 9, 1, 1)
[pid 10 | ppid 9] I am evaluating request #1: "Hacer el sandwich perfecto"
[pid 10 | ppid 9] Dificultad 80: delego en 2 Meeseeks (dificultad 85 cada uno)
  [pid 11 | ppid 10] Hi, I'm Mr. Meeseeks! Look at Meeeee. (11, 10, 2, 1)
  [pid 11 | ppid 10] I am executing request #1 (dificultad 85, 1.0 s)
  [pid 12 | ppid 10] Hi, I'm Mr. Meeseeks! Look at Meeeee. (12, 10, 2, 2)
  [pid 12 | ppid 10] I am executing request #1 (dificultad 85, 1.0 s)
  [pid 11 | ppid 10] Listo! Solicitud #1 completada. Gracias por dejarme servirle.
    Adios!
  [pid 12 | ppid 10] Listo! Solicitud #1 completada. Gracias por dejarme servirle.
    Adios!
[pid 10 | ppid 9] La solicitud la resolvio el Mr. Meeseeks (pid 11, ppid 10, N=2, i=1)
[pid 10 | ppid 9] Solicitud #1 completada. Adios!

================== INFORME DE LA MEESEEKS BOX ==================
Solicitudes: 1   Completadas: 1   No completadas: 0
#    Solicitud                          T.total    T.real  Estado         Meeseeks
1    Hacer el sandwich perfecto         1.0018s   0.0007s  completada            3
T.real = tiempo total sin las esperas simuladas de los intentos (- = no hubo ruta
  ganadora)
================================================================
```

Figura 2. Caso A, Hacer el sandwich perfecto (d = 80) con intentos de 1 s.

La segunda corrida usa Terminar la tarea de SO a tiempo (d = 20). Los 5 delegados reciben d = 23; algunos fallan y llaman a 5 ayudantes cada uno en el nivel 3, pero un hermano lo logra al mismo tiempo, así que los ayudantes reciben STOP y se despiden sin terminar su intento.

```
$ ./meseeks
[pid 18 | ppid 17] Hi, I'm Mr. Meeseeks! Look at Meeeee. (18, 17, 1, 1)
[pid 18 | ppid 17] I am evaluating request #1: "Terminar la tarea de SO a tiempo"
[pid 18 | ppid 17] Dificultad 20: delego en 5 Meeseeks (dificultad 23 cada uno)
  [pid 19 | ppid 18] Hi, I'm Mr. Meeseeks! Look at Meeeee. (19, 18, 2, 1)
  [pid 19 | ppid 18] I am executing request #1 (dificultad 23, 1.0 s)
  [pid 20 | ppid 18] Hi, I'm Mr. Meeseeks! Look at Meeeee. (20, 18, 2, 2)
  [pid 20 | ppid 18] I am executing request #1 (dificultad 23, 1.0 s)
  [pid 21 | ppid 18] Hi, I'm Mr. Meeseeks! Look at Meeeee. (21, 18, 2, 3)
  [pid 21 | ppid 18] I am executing request #1 (dificultad 23, 1.0 s)
  [pid 22 | ppid 18] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22, 18, 2, 4)
  [pid 22 | ppid 18] I am executing request #1 (dificultad 23, 1.0 s)
  [pid 23 | ppid 18] Hi, I'm Mr. Meeseeks! Look at Meeeee. (23, 18, 2, 5)
  [pid 23 | ppid 18] I am executing request #1 (dificultad 23, 1.0 s)
  [pid 19 | ppid 18] Listo! Solicitud #1 completada. Gracias por dejarme servirle.
    Adios!
  [pid 20 | ppid 18] No pude! Invoco 5 ayudantes (dificultad 26)
  [pid 21 | ppid 18] Listo! Solicitud #1 completada. Gracias por dejarme servirle.
    Adios!
  [pid 22 | ppid 18] No pude! Invoco 5 ayudantes (dificultad 26)
    [pid 24 | ppid 20] Hi, I'm Mr. Meeseeks! Look at Meeeee. (24, 20, 3, 1)
    [pid 24 | ppid 20] I am executing request #1 (dificultad 26, 1.0 s)
  [pid 23 | ppid 18] No pude! Invoco 5 ayudantes (dificultad 26)
    [pid 27 | ppid 22] Hi, I'm Mr. Meeseeks! Look at Meeeee. (27, 22, 3, 1)
    [pid 27 | ppid 22] I am executing request #1 (dificultad 26, 1.0 s)
    [pid 29 | ppid 22] Hi, I'm Mr. Meeseeks! Look at Meeeee. (29, 22, 3, 2)
    [pid 29 | ppid 22] I am executing request #1 (dificultad 26, 1.0 s)
[ ... se omiten 36 lineas (salida completa en informe/corridas/A2_tarea_so.txt) ... ]
    [pid 24 | ppid 20] Otro Meeseeks ya lo resolvio. Adios!
    [pid 38 | ppid 23] Hi, I'm Mr. Meeseeks! Look at Meeeee. (38, 23, 3, 5)
    [pid 38 | ppid 23] I am executing request #1 (dificultad 26, 1.0 s)
    [pid 38 | ppid 23] Otro Meeseeks ya lo resolvio. Adios!
  [pid 23 | ppid 18] Otro Meeseeks ya lo resolvio. Adios!
  [pid 20 | ppid 18] Otro Meeseeks ya lo resolvio. Adios!
[pid 18 | ppid 17] La solicitud la resolvio el Mr. Meeseeks (pid 19, ppid 18, N=2,
  i=1)
[pid 18 | ppid 17] Solicitud #1 completada. Adios!

================== INFORME DE LA MEESEEKS BOX ==================
Solicitudes: 1   Completadas: 1   No completadas: 0
#    Solicitud                          T.total    T.real  Estado         Meeseeks
1    Terminar la tarea de SO a tiempo       1.0038s   0.0027s  completada          21
T.real = tiempo total sin las esperas simuladas de los intentos (- = no hubo ruta
  ganadora)
================================================================
```

Figura 3. Caso A, Terminar la tarea de SO a tiempo (d = 20): ayudantes del nivel 3 detenidos con STOP.

La tercera es una solicitud libre, Aprobar Sistemas Operativos, cuya dificultad se dejó a la suerte del Meeseeks: salió 79, por lo que se delegó en 2 Meeseeks con d = 84.

```
$ ./meseeks
[pid 46 | ppid 45] Hi, I'm Mr. Meeseeks! Look at Meeeee. (46, 45, 1, 1)
[pid 46 | ppid 45] I am evaluating request #1: "Aprobar Sistemas Operativos"
[pid 46 | ppid 45] Lo dejo a mi suerte: dificultad 79
[pid 46 | ppid 45] Dificultad 79: delego en 2 Meeseeks (dificultad 84 cada uno)
  [pid 47 | ppid 46] Hi, I'm Mr. Meeseeks! Look at Meeeee. (47, 46, 2, 1)
  [pid 47 | ppid 46] I am executing request #1 (dificultad 84, 1.0 s)
  [pid 48 | ppid 46] Hi, I'm Mr. Meeseeks! Look at Meeeee. (48, 46, 2, 2)
  [pid 48 | ppid 46] I am executing request #1 (dificultad 84, 1.0 s)
  [pid 47 | ppid 46] Listo! Solicitud #1 completada. Gracias por dejarme servirle.
    Adios!
  [pid 48 | ppid 46] Listo! Solicitud #1 completada. Gracias por dejarme servirle.
    Adios!
[pid 46 | ppid 45] La solicitud la resolvio el Mr. Meeseeks (pid 47, ppid 46, N=2,
  i=1)
[pid 46 | ppid 45] Solicitud #1 completada. Adios!

================== INFORME DE LA MEESEEKS BOX ==================
Solicitudes: 1   Completadas: 1   No completadas: 0
#    Solicitud                          T.total    T.real  Estado         Meeseeks
1    Aprobar Sistemas Operativos        1.0017s   0.0006s  completada            3
T.real = tiempo total sin las esperas simuladas de los intentos (- = no hubo ruta
  ganadora)
================================================================
```

Figura 4. Caso A, solicitud libre con dificultad a la suerte del Meeseeks (salió 79).

3.2 Caso B: cálculos

La resta 3 − 8 usa un Meeseeks para la distancia (5) y otro para el signo (−). La división −17 / 5 usa los tres Meeseeks descritos y da cociente −3 y residuo −2, con la convención de C. La operación lógica 1 XOR 1 da 0 después de un intento simulado de 1 s, que no se cuenta en el tiempo real.

```
$ ./meseeks
[pid 840 | ppid 839] Hi, I'm Mr. Meeseeks! Look at Meeeee. (840, 839, 1, 1)
[pid 840 | ppid 839] I am evaluating request #1: "Restar dos numeros"
  [pid 841 | ppid 840] Hi, I'm Mr. Meeseeks! Look at Meeeee. (841, 840, 2, 1)
  [pid 841 | ppid 840] I am executing request #1: distancia entre 3 y 8 = 5
  [pid 842 | ppid 840] Hi, I'm Mr. Meeseeks! Look at Meeeee. (842, 840, 2, 2)
  [pid 842 | ppid 840] I am executing request #1: el signo de 3 - 8 es -
[pid 840 | ppid 839] Resultado: 3 - 8 = -5
[pid 840 | ppid 839] Solicitud #1 completada. Adios!

================== INFORME DE LA MEESEEKS BOX ==================
Solicitudes: 1   Completadas: 1   No completadas: 0
#    Solicitud                          T.total    T.real  Estado         Meeseeks
1    Restar dos numeros                 0.0006s   0.0006s  completada            3
T.real = tiempo total sin las esperas simuladas de los intentos (- = no hubo ruta
  ganadora)
================================================================
```

Figura 5. Caso B, resta 3 − 8 con dos Meeseeks.

```
$ ./meseeks
[pid 848 | ppid 847] Hi, I'm Mr. Meeseeks! Look at Meeeee. (848, 847, 1, 1)
[pid 848 | ppid 847] I am evaluating request #1: "Dividir dos numeros"
  [pid 849 | ppid 848] Hi, I'm Mr. Meeseeks! Look at Meeeee. (849, 848, 2, 1)
  [pid 849 | ppid 848] I am executing request #1: soy el divisor (5), cuento las
    restas
  [pid 851 | ppid 848] Hi, I'm Mr. Meeseeks! Look at Meeeee. (851, 848, 2, 3)
  [pid 850 | ppid 848] Hi, I'm Mr. Meeseeks! Look at Meeeee. (850, 848, 2, 2)
  [pid 850 | ppid 848] I am executing request #1: soy el dividendo (17), resto 5
    mientras quepa
  [pid 851 | ppid 848] I am executing request #1: soy el residuo, sobra -2
[pid 848 | ppid 847] Resultado: -17 / 5 = -3, residuo -2
[pid 848 | ppid 847] Solicitud #1 completada. Adios!

================== INFORME DE LA MEESEEKS BOX ==================
Solicitudes: 1   Completadas: 1   No completadas: 0
#    Solicitud                          T.total    T.real  Estado         Meeseeks
1    Dividir dos numeros                0.0008s   0.0008s  completada            4
T.real = tiempo total sin las esperas simuladas de los intentos (- = no hubo ruta
  ganadora)
================================================================
```

Figura 6. Caso B, división –17 / 5 con divisor, dividendo y residuo.

```
$ ./meseeks
[pid 857 | ppid 856] Hi, I'm Mr. Meeseeks! Look at Meeeee. (857, 856, 1, 1)
[pid 857 | ppid 856] I am evaluating request #1: "O exclusivo (XOR)"
[pid 857 | ppid 856] Dificultad 90: delego en 1 Meeseeks
  [pid 858 | ppid 857] Hi, I'm Mr. Meeseeks! Look at Meeeee. (858, 857, 2, 1)
  [pid 858 | ppid 857] I am executing request #1: O exclusivo (XOR) (1.0 s)
[pid 857 | ppid 856] Resultado: 1 XOR 1 = 0
[pid 857 | ppid 856] Solicitud #1 completada. Adios!

================== INFORME DE LA MEESEEKS BOX ==================
Solicitudes: 1   Completadas: 1   No completadas: 0
#    Solicitud                          T.total    T.real  Estado         Meeseeks
1    O exclusivo (XOR)                  1.0015s   0.0012s  completada            2
T.real = tiempo total sin las esperas simuladas de los intentos (- = no hubo ruta
  ganadora)
================================================================
```

Figura 7. Caso B, 1 XOR 1 (dificultad 90, un solo delegado).

3.3 Caso C: programas externos

date termina con código 0 y la solicitud se completa; false termina con código 1 y la solicitud no se completa. En la tercera corrida se escribieron dos comandos: echo con un mensaje, que funciona, y un programa que no existe, para el cual execvp() falla y el Meeseeks reporta el código 127.

```
$ ./meseeks
[pid 864 | ppid 863] Hi, I'm Mr. Meeseeks! Look at Meeeee. (864, 863, 1, 1)
[pid 864 | ppid 863] I am evaluating request #1: "Mostrar la fecha (date)"
  [pid 865 | ppid 864] Hi, I'm Mr. Meeseeks! Look at Meeeee. (865, 864, 2, 1)
  [pid 865 | ppid 864] I am executing request #1: ejecuto "date"
Wed Oct 14 23:17:39 UTC 2026
  [pid 865 | ppid 864] El programa termino con codigo 0 en 0.001 s
[pid 864 | ppid 863] Codigo 0: el programa termino bien.
[pid 864 | ppid 863] Solicitud #1 completada. Adios!

================== INFORME DE LA MEESEEKS BOX ==================
Solicitudes: 1   Completadas: 1   No completadas: 0
#    Solicitud                          T.total    T.real  Estado         Meeseeks
1    Mostrar la fecha (date)            0.0025s   0.0025s  completada            2
T.real = tiempo total sin las esperas simuladas de los intentos (- = no hubo ruta
  ganadora)
================================================================
```

Figura 8. Caso C, date (código 0, completada).

```
$ ./meseeks
[pid 872 | ppid 871] Hi, I'm Mr. Meeseeks! Look at Meeeee. (872, 871, 1, 1)
[pid 872 | ppid 871] I am evaluating request #1: "Programa que siempre falla (false)"
  [pid 873 | ppid 872] Hi, I'm Mr. Meeseeks! Look at Meeeee. (873, 872, 2, 1)
  [pid 873 | ppid 872] I am executing request #1: ejecuto "false"
  [pid 873 | ppid 872] El programa termino con codigo 1 en 0.001 s
[pid 872 | ppid 871] Codigo 1: el programa fallo.
[pid 872 | ppid 871] Solicitud #1 NO completada. Adios!

================== INFORME DE LA MEESEEKS BOX ==================
Solicitudes: 1   Completadas: 0   No completadas: 1
#    Solicitud                          T.total    T.real  Estado         Meeseeks
1    Programa que siempre falla (false)     0.0012s   0.0012s  no completada         2
T.real = tiempo total sin las esperas simuladas de los intentos (- = no hubo ruta
  ganadora)
================================================================
```

Figura 9. Caso C, false (código 1, no completada).

```
$ ./meseeks
[pid 880 | ppid 879] Hi, I'm Mr. Meeseeks! Look at Meeeee. (880, 879, 1, 1)
[pid 880 | ppid 879] I am evaluating request #1: "Ejecutar: echo Hola desde un
  programa externo"
  [pid 881 | ppid 880] Hi, I'm Mr. Meeseeks! Look at Meeeee. (881, 880, 2, 1)
  [pid 881 | ppid 880] I am executing request #1: ejecuto "echo Hola desde un
    programa externo"
Hola desde un programa externo
  [pid 881 | ppid 880] El programa termino con codigo 0 en 0.001 s
[pid 880 | ppid 879] Codigo 0: el programa termino bien.
[pid 880 | ppid 879] Solicitud #1 completada. Adios!
[pid 883 | ppid 879] Hi, I'm Mr. Meeseeks! Look at Meeeee. (883, 879, 1, 1)
[pid 883 | ppid 879] I am evaluating request #2: "Ejecutar: no_existe"
  [pid 884 | ppid 883] Hi, I'm Mr. Meeseeks! Look at Meeeee. (884, 883, 2, 1)
  [pid 884 | ppid 883] I am executing request #2: ejecuto "no_existe"
no_existe: No such file or directory
  [pid 884 | ppid 883] El programa termino con codigo 127 en 0.000 s
[pid 883 | ppid 879] Codigo 127: el programa fallo.
[pid 883 | ppid 879] Solicitud #2 NO completada. Adios!

================== INFORME DE LA MEESEEKS BOX ==================
Solicitudes: 2   Completadas: 1   No completadas: 1
#    Solicitud                          T.total    T.real  Estado         Meeseeks
1    Ejecutar: echo Hola desde un program   0.0014s   0.0014s  completada           2
2    Ejecutar: no_existe                    0.0008s   0.0008s  no completada        2
T.real = tiempo total sin las esperas simuladas de los intentos (- = no hubo ruta
  ganadora)
================================================================
```

Figura 10. Caso C, comandos escritos por el usuario: echo (0) y un programa inexistente (127).

3.4 Tarea imposible, caos planetario e informe

Para ver el caos planetario sin esperar 5 minutos se configuró un tiempo máximo de 4 s y se pidió Que Jerry deje de ser Jerry (d = 0). Los Meeseeks fallan y se multiplican nivel tras nivel hasta que vence el tiempo; la Box termina al grupo completo, imprime el informe con estado caos y cierra.

```
$ ./meseeks
[pid 54 | ppid 53] Hi, I'm Mr. Meeseeks! Look at Meeeee. (54, 53, 1, 1)
[pid 54 | ppid 53] I am evaluating request #1: "Que Jerry deje de ser Jerry
  (imposible)"
[pid 54 | ppid 53] Dificultad 0: delego en 5 Meeseeks (dificultad 0 cada uno)
  [pid 55 | ppid 54] Hi, I'm Mr. Meeseeks! Look at Meeeee. (55, 54, 2, 1)
  [pid 55 | ppid 54] I am executing request #1 (dificultad 0, 1.0 s)
  [pid 57 | ppid 54] Hi, I'm Mr. Meeseeks! Look at Meeeee. (57, 54, 2, 3)
  [pid 57 | ppid 54] I am executing request #1 (dificultad 0, 1.0 s)
  [pid 56 | ppid 54] Hi, I'm Mr. Meeseeks! Look at Meeeee. (56, 54, 2, 2)
  [pid 56 | ppid 54] I am executing request #1 (dificultad 0, 1.0 s)
  [pid 58 | ppid 54] Hi, I'm Mr. Meeseeks! Look at Meeeee. (58, 54, 2, 4)
  [pid 58 | ppid 54] I am executing request #1 (dificultad 0, 1.0 s)
  [pid 59 | ppid 54] Hi, I'm Mr. Meeseeks! Look at Meeeee. (59, 54, 2, 5)
  [pid 59 | ppid 54] I am executing request #1 (dificultad 0, 1.0 s)
  [pid 57 | ppid 54] No pude! Invoco 5 ayudantes (dificultad 0)
[ ... se omiten 1700 lineas (salida completa en informe/corridas/A4_imposible.txt)
  ... ]
        [pid 833 | ppid 150] Hi, I'm Mr. Meeseeks! Look at Meeeee. (833, 150, 5, 4)
        [pid 833 | ppid 150] I am executing request #1 (dificultad 0, 1.0 s)
        [pid 834 | ppid 150] Hi, I'm Mr. Meeseeks! Look at Meeeee. (834, 150, 5, 5)
        [pid 834 | ppid 150] I am executing request #1 (dificultad 0, 1.0 s)
*** CAOS PLANETARIO *** Se superaron los 4 s. La Box termina a todos los Meeseeks.
El caos planetario obliga a cerrar la Meeseeks Box.

================== INFORME DE LA MEESEEKS BOX ==================
Solicitudes: 1   Completadas: 0   No completadas: 1
#    Solicitud                          T.total    T.real  Estado         Meeseeks
1    Que Jerry deje de ser Jerry (imposib   4.0262s         -  caos               781
T.real = tiempo total sin las esperas simuladas de los intentos (- = no hubo ruta
  ganadora)
================================================================
```

Figura 11. Tarea imposible con tiempo máximo de 4 s: caos planetario con 781 Meeseeks creados.

Por último, el informe de una sesión con cinco solicitudes de los tres casos, que la Box imprime al despedirse:

```
$ ./meseeks
Adios!
================== INFORME DE LA MEESEEKS BOX ==================
Solicitudes: 5   Completadas: 5   No completadas: 0
#    Solicitud                          T.total    T.real  Estado         Meeseeks
1    Encontrar las llaves de la casa    0.5013s   0.0006s  completada            2
2    Aprender a tocar guitarra          0.5013s   0.0007s  completada            3
3    Multiplicar dos numeros            0.0007s   0.0007s  completada            3
4    O exclusivo (XOR)                  0.5009s   0.0007s  completada            2
5    Listar esta carpeta (ls -l)        0.0021s   0.0021s  completada            2
T.real = tiempo total sin las esperas simuladas de los intentos (- = no hubo ruta ganadora)
================================================================
```

Figura 12. Informe de la Box al despedirse después de cinco solicitudes.

4. Experimentos y análisis

Los experimentos se automatizaron con un guion en Python (informe/experimentos.py) que alimenta la Box por la entrada estándar y lee la tabla del informe; los datos crudos están en informe/experimentos.csv. Se plantearon cuatro experimentos, cada uno cambiando una sola variable:

- **E1:** dificultad d de 100 a 0 con intentos fijos de 0.5 s, 20 repeticiones por valor.
- **E2:** dificultades 80, 40 y 20 con intentos al azar de 0.5 a 5 s, 10 repeticiones.
- **E3:** casos B y C (10 repeticiones; 5 para la lógica, con intento de 0.5 s).
- **E4:** d = 1 y d = 0 con un tiempo máximo de 3 s en modo de seguir esperando, y d = 0 con caos.

4.1 E1: efecto de la dificultad

Tabla 2. E1, intentos fijos de 0.5 s (20 repeticiones por dificultad).

d	T. total mediana (s)	T. total máx. (s)	T. real mediana (ms)	Meeseeks mediana	Meeseeks máx.	En 1 ronda	Completadas
100	0.501	0.503	0.80	2	2	20/20	20/20
80	0.501	0.503	0.80	3	5	20/20	20/20
60	0.502	1.005	1.45	5	9	15/20	20/20
40	0.503	0.506	2.40	16	26	20/20	20/20
20	0.505	1.023	3.80	26	146	15/20	20/20
1	1.020	1.619	14.25	143.5	716	4/20	20/20
0	2.821	2.919	-	3906	3906	0/20	0/20

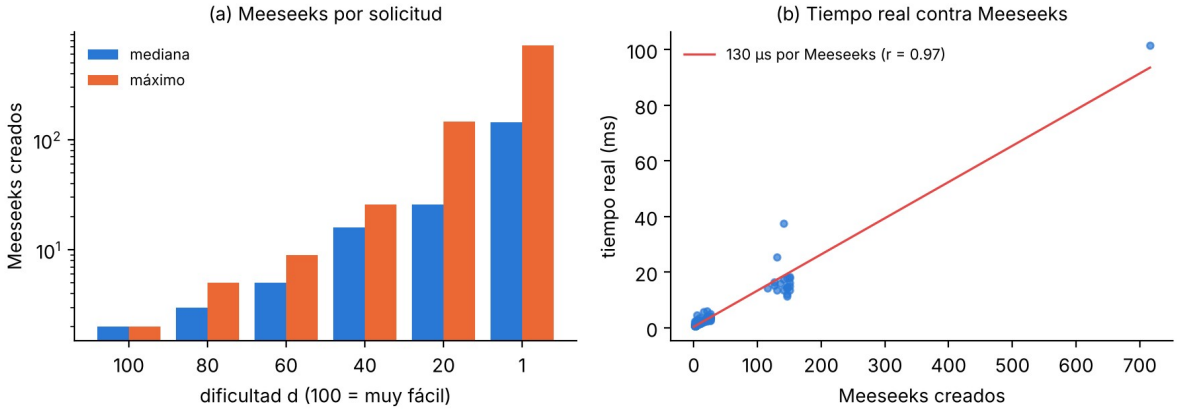


Figura 13. E1: (a) Meeseeks creados según la dificultad, en escala logarítmica; (b) tiempo real contra cantidad de Meeseeks.

Con intentos fijos, el tiempo total es un múltiplo de 0.5 s: 0.5 s si la solicitud se resuelve en la primera ronda de delegados, 1 s si hace falta una ronda de ayudantes y así sucesivamente. La fracción de corridas resueltas en una ronda coincide con la probabilidad teórica de la Tabla 1 (por ejemplo, 4 de 20 con d = 1 frente a 18 %, y 15 de 20 con d = 20 frente a 73 %). Un resultado que no es obvio es que d = 40 se resolvió siempre en una ronda y d = 60 no: con d ≤ 45 se llama a 5 delegados en lugar de 2, y tener más Meeseeks intentando en paralelo pesa más que la diferencia de dificultad.

El tiempo real crece de forma lineal con la cantidad de Meeseeks: el ajuste da 130 µs por Meeseeks (r = 0.97), que es lo que cuesta un fork(), dos pipes, el registro con semáforo y los mensajes. La cantidad de Meeseeks es muy variable cuando la tarea es difícil (con d = 1 va de 26 a 716). Con d = 0 siempre se crean 3906 Meeseeks y el tiempo es de 2.82 s: 5 niveles de intentos de 0.5 s más unos 0.32 s de crear y terminar procesos, unos 82 µs por Meeseeks, consistente con el ajuste.

4.2 E2: duración aleatoria de los intentos

Tabla 3. E2, intentos al azar entre 0.5 y 5 s (10 repeticiones).

d	Delegados	T. total mediana (s)	Rango (s)	T. real mediana (ms)	Meeseeks mediana	Completadas
80	2	1.92	1.36 a 4.15	0.95	3	10/10
40	5	1.11	0.58 a 1.92	1.40	6	10/10
20	5	2.45	0.94 a 3.71	1.55	8.5	10/10

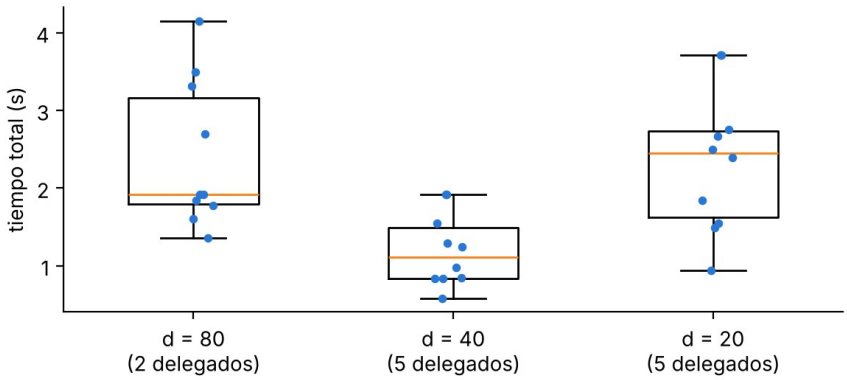


Figura 14. E2: tiempo total de cada repetición según la dificultad.

Con intentos al azar, la solicitud termina cuando el primer Meeseeks que lo logra acaba su intento, de modo que el tiempo total es aproximadamente el mínimo de varias duraciones uniformes. Por eso d = 40, con 5 delegados, tuvo una mediana de 1.11 s, menor que la de d = 80 (1.92 s), que solo tiene 2. Con d = 20 la mediana sube a 2.45 s porque a veces hace falta una segunda ronda. El tiempo real, en cambio, se mantiene en el orden de 1 a 2 ms en las tres dificultades: el tiempo que ve el usuario lo domina la simulación y no el sistema operativo.

4.3 E3: casos B y C

Tabla 4. E3, casos B y C (medianas).

Solicitud	Meeseeks	T. total mediana (ms)	T. real mediana (ms)	Completadas
B, suma 12 + 30	2	0.50	0.50	10/10
B, resta 12 − 30	3	0.55	0.55	10/10
B, multiplicación 12 × 30	3	0.60	0.60	10/10
B, división 300 / 7	4	0.80	0.80	10/10
B, 1 Y 0 (intento de 0.5 s)	2	500.80	0.70	5/5

C, date	2	1.35	1.35	10/10
C, ls -l	2	1.85	1.85	10/10
C, false	2	1.10	1.10	0/10

En el caso B el tiempo real crece con la cantidad de Meeseeks que cooperan: la suma y la resta cuestan alrededor de 0.5 ms, la multiplicación un poco más por las 30 iteraciones que se pasan por pipe y la división unos 0.8 ms con 4 Meeseeks. Calcular es despreciable frente a crear procesos. En el caso C se suma un fork() y un exec() más, por lo que date y ls -l rondan 1.5 a 2 ms; false cuesta lo mismo que date pero termina con código 1 y se reporta como no completada. En la lógica el tiempo total es el del intento simulado y el tiempo real es igual al de una suma.

4.4 E4: tiempo máximo y tiempo adicional

Tabla 5. E4, tiempo máximo de 3 s con intentos al azar.

Configuración	Corridas	T. total (s)	Tiempo adicional (s)	Meeseeks	Completadas
d = 1, seguir esperando	3	3.4, 5.4, 2.7	0.4, 2.4, 0.0	41, 61, 46	3/3
d = 0, seguir esperando	3	23.1, 22.1, 22.1	20.1, 19.1, 19.1	3906, 3906, 3906	0/3
d = 0, caos	1	3.0	0.0	46	0/1

El enunciado pregunta qué pasa si una solicitud muy difícil no se cumple en el tiempo máximo y cuánto tiempo adicional necesita. Con el tiempo máximo de 300 s ninguna solicitud lo alcanzó en ninguna prueba, así que se bajó a 3 s. Con d = 1, 2 de las 3 corridas superaron los 3 s y aun así se completaron, con hasta 2.4 s adicionales: la dilución hace que en pocos niveles la tarea sea realizable. Con d = 0 la solicitud nunca se completa, pero tampoco crece sin límite: a los 22 s se agotan los 3906 Meeseeks permitidos por el límite de nivel y termina como no completada; el tiempo adicional es finito pero no sirve de nada. Con caos, la Box corta exactamente a los 3 s y termina todos los procesos.

5. Conclusiones

- Crear y coordinar un Meeseeks cuesta unos 130 µs de tiempo real (fork, pipes, semáforo y mensajes); frente a esperas simuladas de 0.5 a 5 s es despreciable, por eso el tiempo total depende casi solo de la simulación.
- La cantidad de Meeseeks trabajando en paralelo influye más que la dificultad: d = 40, con 5 delegados, se resolvió antes y con más seguridad que d = 60, con 2.
- La dilución inspirada en el añejamiento garantiza que toda tarea con d ≥ 1 termine en pocos niveles, mientras que d = 0 queda contenida por el límite N ≥ 6 en 3906 procesos y unos 25 s como máximo.
- Por lo anterior, con la base de 5 minutos el caos planetario solo aparece si se configura un tiempo máximo menor que la duración posible de una solicitud; cuando ocurre, el grupo de procesos permite terminar a todos los Meeseeks de una vez sin dejar procesos huérfanos.
- Los pipes por hijo con poll() permiten que el Resultado suba y el STOP baje en cualquier momento, y el semáforo en memoria compartida basta para los contadores, ya que es el único recurso que comparten.

Referencias

[1] A. Silberschatz, P. B. Galvin y G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] N. Matthew y R. Stones, Beginning Linux Programming, 4th ed. Indianapolis, IN, USA: Wiley, 2008, caps. 13 y 14.

[3] M. Kerrisk, The Linux Programming Interface. San Francisco, CA, USA: No Starch Press, 2010.

[4] Linux man-pages project, "fork(2), pipe(2), poll(2), execvp(3), sem_init(3), mmap(2), alarm(2)". [En línea]. Disponible: https://man7.org/linux/man-pages/

Apéndice: código fuente

Archivo meseeks.c, compilado con gcc -Wall -pthread -o meseeks meseeks.c.

```
/*
 * meseeks.c - Tarea 3, Box de Mr. Meeseeks
 *
 * Instituto Tecnologico de Costa Rica, Campus Tecnologico Central Cartago
 * Escuela de Ingenieria en Computacion, II Semestre 2026
 * Curso:      IC-6600 Principios de Sistemas Operativos
 * Profesor:   Esteban Arias Mendez
 * Estudiante: Sergio Andres Zuniga Castillo, carne 2024146884
 *
 * Flujo:  Usuario -(menu)-> Box -fork()-> Meeseeks evaluador (uno por solicitud)
 *         -fork() + pipes-> Meeseeks delegados que resuelven la solicitud.
 *
 * La solicitud baja por pipe; el resultado sube por pipe. Cuando un delegado lo
 * logra, su padre avisa por pipe a los hermanos para que terminen.
 * Cada opcion del menu tiene su propia funcion, que ejecuta el evaluador.
 * Documentacion de cada funcion: meseeks.md
 *
 * Compilar: gcc -Wall -pthread -o meseeks meseeks.c
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>
#include <errno.h>
#include <signal.h>
#include <time.h>
#include <unistd.h>
#include <fcntl.h>
#include <poll.h>
#include <semaphore.h>
#include <sys/mman.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_NIVELES  32
#define MAX_HIJOS    10
#define MAX_TAREAS   100
#define NIVEL_LIMITE 6     /* un Meeseeks con N >= 6 ya no puede pedir ayuda */
#define STOP         1     /* mensaje padre -> hijo: ya se resolvio, terminen */

/* ========================================================================== */
/*  Contadores compartidos (memoria compartida + semaforo)                    */
/* ========================================================================== */

/* instancias[N] = cuantos Meeseeks se han creado en el nivel N.
 * El semaforo mutex protege el acceso a los contadores. */
typedef struct {
    sem_t mutex;
    int nivel_max;
    int instancias[MAX_NIVELES + 1];
    double espera;     /* segundos simulados en la ruta que resolvio la solicitud */
} Contadores;

static Contadores *cont;   /* apunta a la memoria compartida */

void contadores_crear(void)
{
    cont = mmap(NULL, sizeof(Contadores), PROT_READ | PROT_WRITE,
                MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (cont == MAP_FAILED) {
```

```
        perror("mmap");
        exit(1);
    }
    sem_init(&cont->mutex, 1, 1);   /* 1 = compartido entre procesos */
    cont->nivel_max = 0;
    for (int n = 0; n <= MAX_NIVELES; n++)
        cont->instancias[n] = 0;
}

void contadores_reiniciar(void)
{
    sem_wait(&cont->mutex);
    cont->nivel_max = 0;
    cont->espera = 0;
    for (int n = 0; n <= MAX_NIVELES; n++)
        cont->instancias[n] = 0;
    sem_post(&cont->mutex);
}

/* Registra un Meeseeks nuevo: su N es el nivel del padre + 1.
 * Seccion critica con el semaforo. Devuelve N. */
int registrar_meeseeks(int nivel_padre)
{
    int N = nivel_padre + 1;
    if (N > MAX_NIVELES)
        N = MAX_NIVELES;
    sem_wait(&cont->mutex);
    cont->instancias[N]++;
    if (N > cont->nivel_max)
        cont->nivel_max = N;
    sem_post(&cont->mutex);
    return N;
}

int contadores_total(void)
{
    int total = 0;
    sem_wait(&cont->mutex);
    for (int n = 1; n <= MAX_NIVELES; n++)
        total += cont->instancias[n];
    sem_post(&cont->mutex);
    return total;
}

/* El evaluador anota cuanto se espero en la simulacion (para el tiempo real) */
void reportar_espera(double segundos)
{
    sem_wait(&cont->mutex);
    cont->espera = segundos;
    sem_post(&cont->mutex);
}

double espera_reportada(void)
{
    sem_wait(&cont->mutex);
    double e = cont->espera;
    sem_post(&cont->mutex);
    return e;
}

void contadores_destruir(void)
{
    sem_destroy(&cont->mutex);
    munmap(cont, sizeof(Contadores));
}
```

```
/* ========================================================================== */
/*  Tipos, opciones del menu y utilidades                                     */
/* ========================================================================== */

/* Lo que viaja por el pipe del evaluador a sus delegados */
typedef struct {
    int solicitud;      /* numero de solicitud (#k)                    */
    char texto[64];     /* lo que se pidio                             */
    int dificultad;     /* 0..100, 100 = muy facil; -1 = a la suerte   */
    int duracion_ms;    /* duracion de cada intento; 0 = al azar       */
    int a, b;           /* operandos (aritmetica y logica)             */
    char operacion;     /* operacion logica: '&', '|', '^', '!'        */
    char comando[128];  /* programa por ejecutar                       */
} Tarea;

/* Lo que sube por el pipe de un hijo a su padre */
typedef struct {
    int resuelto;        /* 1 = alguien lo logro                         */
    int pid, ppid, N, i; /* quien lo logro                               */
    double espera;       /* s simulados en la ruta del evaluador al que lo logro */
} Resultado;

/* Que datos le pide la Box al usuario antes de crear al evaluador */
enum { PIDE_NADA, PIDE_DOS_NUMEROS, PIDE_DOS_BOOLEANOS, PIDE_UN_BOOLEANO, PIDE_TEXTO,
       PIDE_COMANDO, PIDE_CONFIG };

typedef struct Opcion Opcion;
typedef struct Menu Menu;

struct Opcion {
    int id;
    const char *nombre;
    int dificultad;                                 /* -1 si no aplica          */
    int pide;                                       /* ver enum PIDE_*          */
    int (*funcion)(const Opcion *op, Tarea t);      /* la ejecuta el evaluador;
                                                       1 = completada           */
    const Menu *submenu;                            /* != NULL: abre un submenu */
};

struct Menu {
    const char *titulo;
    const Opcion *opciones;
    int n;
};

int evaluar_texto(const Opcion *op, Tarea t);
int evaluar_suma(const Opcion *op, Tarea t);
int evaluar_resta(const Opcion *op, Tarea t);
int evaluar_multiplicacion(const Opcion *op, Tarea t);
int evaluar_division(const Opcion *op, Tarea t);
int evaluar_y(const Opcion *op, Tarea t);
int evaluar_o(const Opcion *op, Tarea t);
int evaluar_xor(const Opcion *op, Tarea t);
int evaluar_no(const Opcion *op, Tarea t);
int evaluar_fecha(const Opcion *op, Tarea t);
int evaluar_listar(const Opcion *op, Tarea t);
int evaluar_falla(const Opcion *op, Tarea t);
int evaluar_comando(const Opcion *op, Tarea t);

#define CANTIDAD(arr) (int)(sizeof(arr) / sizeof(arr[0]))

/* Submenu general: solicitudes de texto (dificultad: 100 = muy facil, 0 = imposible) */
static const Opcion opciones_general[] = {
    { 1, "Encontrar las llaves de la casa",   100, PIDE_NADA,  evaluar_texto, NULL },
```

```
    { 2, "Hacer el sandwich perfecto",         80, PIDE_NADA,  evaluar_texto, NULL },
    { 3, "Aprender a tocar guitarra",          60, PIDE_NADA,  evaluar_texto, NULL },
    { 4, "Ganar un torneo de ajedrez",         40, PIDE_NADA,  evaluar_texto, NULL },
    { 5, "Terminar la tarea de SO a tiempo",   20, PIDE_NADA,  evaluar_texto, NULL },
    { 6, "Solicitud libre",                    -1, PIDE_TEXTO, evaluar_texto, NULL },
    { 7, "Que Jerry deje de ser Jerry (imposible)", 0, PIDE_NADA, evaluar_texto, NULL },
};
static const Menu menu_general = { "General", opciones_general,
CANTIDAD(opciones_general) };

/* Submenu de aritmetica */
static const Opcion opciones_aritmetica[] = {
    { 1, "Sumar dos numeros",       -1, PIDE_DOS_NUMEROS, evaluar_suma,           NULL },
    { 2, "Restar dos numeros",      -1, PIDE_DOS_NUMEROS, evaluar_resta,          NULL },
    { 3, "Multiplicar dos numeros", -1, PIDE_DOS_NUMEROS, evaluar_multiplicacion, NULL },
    { 4, "Dividir dos numeros",     -1, PIDE_DOS_NUMEROS, evaluar_division,       NULL },
};
static const Menu menu_aritmetica = { "Aritmetica", opciones_aritmetica,
CANTIDAD(opciones_aritmetica) };

/* Submenu de logica (valores 0 = falso, 1 = verdadero) */
static const Opcion opciones_logica[] = {
    { 1, "Y (AND)",  90, PIDE_DOS_BOOLEANOS, evaluar_y,   NULL },
    { 2, "O (OR)",   90, PIDE_DOS_BOOLEANOS, evaluar_o,   NULL },
    { 3, "O exclusivo (XOR)", 90, PIDE_DOS_BOOLEANOS, evaluar_xor, NULL },
    { 4, "NO (NOT)", 90, PIDE_UN_BOOLEANO,   evaluar_no,  NULL },
};
static const Menu menu_logica = { "Logica", opciones_logica, CANTIDAD(opciones_logica) };

/* Submenu de programas externos (fork + exec) */
static const Opcion opciones_programas[] = {
    { 1, "Mostrar la fecha (date)",          -1, PIDE_NADA,    evaluar_fecha,   NULL },
    { 2, "Listar esta carpeta (ls -l)",      -1, PIDE_NADA,    evaluar_listar,  NULL },
    { 3, "Programa que siempre falla (false)", -1, PIDE_NADA,  evaluar_falla,   NULL },
    { 4, "Escribir un comando",              -1, PIDE_COMANDO, evaluar_comando, NULL },
};
static const Menu menu_programas = { "Programas", opciones_programas,
CANTIDAD(opciones_programas) };

/* Menu principal */
static const Opcion opciones_principal[] = {
    { 1, "General...",       -1, PIDE_NADA,   NULL, &menu_general },
    { 2, "Aritmetica...",    -1, PIDE_NADA,   NULL, &menu_aritmetica },
    { 3, "Logica...",        -1, PIDE_NADA,   NULL, &menu_logica },
    { 4, "Programas...",     -1, PIDE_NADA,   NULL, &menu_programas },
    { 5, "Configuracion...", -1, PIDE_CONFIG, NULL, NULL },
};
static const Menu menu_principal = { "Meeseeks Box", opciones_principal,
CANTIDAD(opciones_principal) };

/* Identidad del proceso actual (cada proceso tiene su propia copia) */
static int mi_nivel = 0;      /* N (0 = la Box)            */
static int mi_instancia = 0;  /* i: numero de hijo de su padre */

double ahora(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

void enviar_int(int fd, int x)     { write(fd, &x, sizeof x); }
int  recibir_int(int fd)           { int x = 0; read(fd, &x, sizeof x); return x; }
void enviar_tarea(int fd, Tarea t) { write(fd, &t, sizeof t); }
```

```
Tarea recibir_tarea(int fd)        { Tarea t; memset(&t, 0, sizeof t); read(fd, &t, sizeof
t); return t; }
void enviar_resultado(int fd, Resultado r) { write(fd, &r, sizeof r); }
void enviar_double(int fd, double x) { write(fd, &x, sizeof x); }
double recibir_double(int fd)      { double x = 0; read(fd, &x, sizeof x); return x; }

/* Mensaje de un Meeseeks: siempre con su pid y ppid, sangrado segun su nivel */
void decir(const char *fmt, ...)
{
    char msg[512], linea[600];
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(msg, sizeof msg, fmt, ap);
    va_end(ap);
    int n = snprintf(linea, sizeof linea, "%*s[pid %d | ppid %d] %s\n",
                     (mi_nivel > 1 ? mi_nivel - 1 : 0) * 2, "", getpid(), getppid(), msg);
    write(STDOUT_FILENO, linea, n);
}

/* Todo Meeseeks llama a esta funcion al nacer: se registra (N) y saluda */
void presentarse(int nivel_padre)
{
    mi_nivel = registrar_meeseeks(nivel_padre);
    srand(getpid() ^ time(NULL));
    decir("Hi, I'm Mr. Meeseeks! Look at Meeeee. (%d, %d, %d, %d)",
          getpid(), getppid(), mi_nivel, mi_instancia);
}

/* Crea un Meeseeks hijo numero 'instancia' (su i) que ejecuta 'cuerpo' con
 * los descriptores 'fd'. Devuelve el pid del hijo. */
pid_t crear_meeseeks(void (*cuerpo)(int nivel_padre, const int *fd),
                     int nivel_padre, int instancia, const int *fd)
{
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    }
    if (pid == 0) {
        mi_instancia = instancia;
        cuerpo(nivel_padre, fd);
        exit(0);
    }
    return pid;
}

/* Crea el pipe para mandar la tarea a un Meeseeks y el de su respuesta */
void pipes_tarea(int bajada[2], int subida[2])
{
    pipe(bajada);
    pipe(subida);
}

/* ========================================================================= */
/*  General: solicitudes de texto (simuladas)                                */
/* ========================================================================= */

/* Un hijo visto desde su padre */
typedef struct {
    pid_t pid;
    int baj;     /* para mandarle la tarea y el STOP */
    int sub;     /* para recibir su Resultado        */
    int vivo;    /* todavia no ha respondido         */
} Hijo;
```

```
/* Ayudantes segun las instrucciones */
int ayudantes_necesarios(int d)
{
    if (d > 85) return 1;
    if (d > 45) return 2;
    return 5;
}

/* Al repartir la tarea entre k ayudantes, a cada uno le toca mas facil */
int diluir(int d, int k)
{
    if (d == 0)
        return 0;                  /* lo imposible sigue siendo imposible */
    return d + (100 - d) / (k * k);
}

void delegado_texto(int nivel_padre, const int *fd);

/* Crea k delegados (i = 1..k) y le manda la tarea a cada uno por su pipe */
void crear_delegados(Tarea t, int k, Hijo h[])
{
    for (int j = 0; j < k; j++) {
        int baj[2], sub[2];
        pipes_tarea(baj, sub);
        int fd[] = { baj[0], sub[1], baj[1], sub[0] };
        h[j].pid = crear_meeseeks(delegado_texto, mi_nivel, j + 1, fd);
        close(baj[0]);
        close(sub[1]);
        h[j].baj = baj[1];
        h[j].sub = sub[0];
        h[j].vivo = 1;
        enviar_tarea(h[j].baj, t);            /* la solicitud viaja por pipe */
    }
}

/* Le avisa por pipe a los hijos que siguen trabajando (menos 'excepto') */
void detener_hijos(Hijo h[], int k, int excepto)
{
    for (int j = 0; j < k; j++)
        if (j != excepto && h[j].vivo)
            enviar_int(h[j].baj, STOP);
}

/* Espera a que algun hijo resuelva o a que todos fallen. Si fd_padre >= 0
 * tambien escucha al padre: si manda STOP, detiene a todos y *detenido = 1.
 * Al final espera (waitpid) a todos los hijos. */
Resultado supervisar(Hijo h[], int k, int fd_padre, int *detenido)
{
    Resultado r;
    struct pollfd pf[MAX_HIJOS + 1];
    int pendientes = k;

    memset(&r, 0, sizeof r);
    *detenido = 0;
    pf[0].fd = fd_padre;                      /* -1: poll lo ignora */
    pf[0].events = POLLIN;
    for (int j = 0; j < k; j++) {
        pf[j + 1].fd = h[j].sub;
        pf[j + 1].events = POLLIN;
    }

    while (pendientes > 0) {
        if (poll(pf, k + 1, -1) < 0)
            continue;
        if (fd_padre >= 0 && pf[0].revents) {     /* el padre ordena parar */
```

```
            recibir_int(fd_padre);
            *detenido = 1;
            detener_hijos(h, k, -1);
            break;
        }
        for (int j = 0; j < k; j++) {
            if (!h[j].vivo || !pf[j + 1].revents)
                continue;
            Resultado m;
            int n = read(h[j].sub, &m, sizeof m);
            h[j].vivo = 0;
            pf[j + 1].fd = -1;
            pendientes--;
            if (n == (int)sizeof m && m.resuelto) {
                r = m;                              /* alguien lo logro */
                detener_hijos(h, k, j);             /* avisar a los hermanos */
                pendientes = 0;
                break;
            }
        }
    }
    for (int j = 0; j < k; j++) {
        waitpid(h[j].pid, NULL, 0);
        close(h[j].baj);
        close(h[j].sub);
    }
    return r;
}

/* Cuerpo de un delegado: recibe la tarea, la intenta y si falla pide ayuda.
 * fd[0] = de su padre, fd[1] = a su padre. */
void delegado_texto(int nivel_padre, const int *fd)
{
    Resultado r;
    int detenido;
    close(fd[2]);
    close(fd[3]);
    Tarea t = recibir_tarea(fd[0]);
    presentarse(nivel_padre);
    memset(&r, 0, sizeof r);

    int ms = t.duracion_ms > 0 ? t.duracion_ms : 500 + rand() % 4501;
    decir("I am executing request #%d (dificultad %d, %.1f s)", t.solicitud, t.dificultad,
ms / 1000.0);

    /* Trabaja, pero atento por si su padre le avisa que ya se resolvio */
    struct pollfd pf = { fd[0], POLLIN, 0 };
    double t0 = ahora();
    if (poll(&pf, 1, ms) > 0) {
        recibir_int(fd[0]);
        decir("Otro Meeseeks ya lo resolvio. Adios!");
        exit(0);
    }
    double mi_espera = ahora() - t0;          /* tiempo simulado de este intento */

    if (t.dificultad > 85 || rand() % 100 < t.dificultad) {
        r.resuelto = 1;
        r.pid = getpid();
        r.ppid = getppid();
        r.N = mi_nivel;
        r.i = mi_instancia;
        r.espera = mi_espera;
        decir("Listo! Solicitud #%d completada. Gracias por dejarme servirle. Adios!",
t.solicitud);
        enviar_resultado(fd[1], r);
```

```
        exit(0);
    }

    if (mi_nivel >= NIVEL_LIMITE) {
        decir("No pude, y en el nivel %d ya no puedo pedir ayuda. Adios!", mi_nivel);
        enviar_resultado(fd[1], r);
        exit(0);
    }

    Hijo h[MAX_HIJOS];
    int k = ayudantes_necesarios(t.dificultad);
    t.dificultad = diluir(t.dificultad, k);
    decir("No pude! Invoco %d ayudantes (dificultad %d)", k, t.dificultad);
    crear_delegados(t, k, h);
    r = supervisar(h, k, fd[0], &detenido);

    if (detenido)
        decir("Otro Meeseeks ya lo resolvio. Adios!");
    else if (r.resuelto)
        decir("Mi ayudante (%d, %d, %d, %d) lo logro, le aviso a mi padre. Adios!", r.pid,
r.ppid, r.N, r.i);
    else
        decir("Ni mis ayudantes pudieron. Adios!");
    r.espera += mi_espera;                    /* la ruta pasa por este intento */
    if (!detenido)
        enviar_resultado(fd[1], r);
    exit(0);
}

int evaluar_texto(const Opcion *op, Tarea t)
{
    Hijo h[MAX_HIJOS];
    int detenido;
    (void)op;
    if (t.dificultad < 0) {                     /* caso 2: a la suerte del Meeseeks */
        t.dificultad = rand() % 101;
        decir("Lo dejo a mi suerte: dificultad %d", t.dificultad);
    }
    int original = t.dificultad;
    int k = ayudantes_necesarios(t.dificultad);
    t.dificultad = diluir(t.dificultad, k);
    decir("Dificultad %d: delego en %d Meeseeks (dificultad %d cada uno)", original, k,
t.dificultad);

    crear_delegados(t, k, h);
    Resultado r = supervisar(h, k, -1, &detenido);
    reportar_espera(r.resuelto ? r.espera : -1);   /* -1: no hubo ruta ganadora */
    if (r.resuelto)
        decir("La solicitud la resolvio el Mr. Meeseeks (pid %d, ppid %d, N=%d, i=%d)",
              r.pid, r.ppid, r.N, r.i);
    else
        decir("Ningun Meeseeks pudo con la solicitud.");
    return r.resuelto;
}

/* ========================================================================== */
/*  Aritmetica. fd[0] = recibe la tarea, fd[1] = envia resultado,             */
/*  fd[2] / fd[3] (y fd[4]) = pipes entre hermanos (si aplica)                */
/* ========================================================================== */

/* Suma: un Meeseeks toma ambos numeros y los suma */
void m_sumador(int nivel_padre, const int *fd)
{
    Tarea t = recibir_tarea(fd[0]);
    presentarse(nivel_padre);
```

```
    decir("I am executing request #%d: %d + %d", t.solicitud, t.a, t.b);
    enviar_int(fd[1], t.a + t.b);
}

/* Resta: uno calcula la distancia entre los numeros ... */
void m_distancia(int nivel_padre, const int *fd)
{
    Tarea t = recibir_tarea(fd[0]);
    presentarse(nivel_padre);
    int dist = t.a > t.b ? t.a - t.b : t.b - t.a;
    decir("I am executing request #%d: distancia entre %d y %d = %d", t.solicitud, t.a,
t.b, dist);
    enviar_int(fd[1], dist);
}

/* ... y otro determina el signo */
void m_signo(int nivel_padre, const int *fd)
{
    Tarea t = recibir_tarea(fd[0]);
    presentarse(nivel_padre);
    int signo = t.a >= t.b ? 1 : -1;
    decir("I am executing request #%d: el signo de %d - %d es %c",
          t.solicitud, t.a, t.b, signo > 0 ? '+' : '-');
    enviar_int(fd[1], signo);
}

/* Multiplicacion: el contador mantiene el multiplicador y le resta 1 por
 * iteracion; por cada iteracion avisa al acumulador (+1 o -1), 0 = fin */
void m_contador(int nivel_padre, const int *fd)
{
    Tarea t = recibir_tarea(fd[0]);
    presentarse(nivel_padre);
    int restante = t.b < 0 ? -t.b : t.b;
    int paso = t.b < 0 ? -1 : 1;
    decir("I am executing request #%d: cuento %d iteraciones", t.solicitud, restante);
    while (restante > 0) {
        enviar_int(fd[3], paso);
        restante--;
    }
    enviar_int(fd[3], 0);
}

/* El acumulador suma (o resta) el multiplicando en cada iteracion */
void m_acumulador(int nivel_padre, const int *fd)
{
    int paso, acc = 0;
    Tarea t = recibir_tarea(fd[0]);
    presentarse(nivel_padre);
    decir("I am executing request #%d: acumulo %d por iteracion", t.solicitud, t.a);
    while ((paso = recibir_int(fd[2])) != 0)
        acc += paso > 0 ? t.a : -t.a;
    enviar_int(fd[1], acc);
}

/* Division: el divisor le pasa el divisor al dividendo y cuenta cuantas
 * veces cupo (cociente) */
void m_divisor(int nivel_padre, const int *fd)
{
    int veces = 0;
    Tarea t = recibir_tarea(fd[0]);
    presentarse(nivel_padre);
    int divisor = t.b < 0 ? -t.b : t.b;
    decir("I am executing request #%d: soy el divisor (%d), cuento las restas",
t.solicitud, divisor);
    enviar_int(fd[3], divisor);
```

```
    while (recibir_int(fd[2]) != 0)
        veces++;
    enviar_int(fd[1], veces);
}

/* El dividendo le resta el divisor mientras quepa, avisa cada resta al
 * divisor y le pasa lo que sobra al residuo. fd[4] = pipe al residuo */
void m_dividendo(int nivel_padre, const int *fd)
{
    Tarea t = recibir_tarea(fd[0]);
    presentarse(nivel_padre);
    int resto = t.a < 0 ? -t.a : t.a;
    int divisor = recibir_int(fd[2]);
    decir("I am executing request #%d: soy el dividendo (%d), resto %d mientras quepa",
          t.solicitud, resto, divisor);
    while (resto >= divisor) {
        resto -= divisor;
        enviar_int(fd[3], 1);
    }
    enviar_int(fd[3], 0);
    enviar_int(fd[4], resto);
}

/* El residuo recibe lo que sobro y le pone el signo del dividendo */
void m_residuo(int nivel_padre, const int *fd)
{
    Tarea t = recibir_tarea(fd[0]);
    presentarse(nivel_padre);
    int resto = recibir_int(fd[2]);
    if (t.a < 0)
        resto = -resto;
    decir("I am executing request #%d: soy el residuo, sobra %d", t.solicitud, resto);
    enviar_int(fd[1], resto);
}

int evaluar_suma(const Opcion *op, Tarea t)
{
    int baj[2], sub[2];
    (void)op;
    pipes_tarea(baj, sub);
    int fd[] = { baj[0], sub[1] };
    crear_meeseeks(m_sumador, mi_nivel, 1, fd);
    enviar_tarea(baj[1], t);
    int r = recibir_int(sub[0]);
    while (wait(NULL) > 0);
    decir("Resultado: %d + %d = %d", t.a, t.b, r);
    return 1;
}

int evaluar_resta(const Opcion *op, Tarea t)
{
    int b1[2], s1[2], b2[2], s2[2];
    (void)op;
    pipes_tarea(b1, s1);
    pipes_tarea(b2, s2);
    int fd1[] = { b1[0], s1[1] };
    int fd2[] = { b2[0], s2[1] };
    crear_meeseeks(m_distancia, mi_nivel, 1, fd1);
    crear_meeseeks(m_signo, mi_nivel, 2, fd2);
    enviar_tarea(b1[1], t);
    enviar_tarea(b2[1], t);
    int dist = recibir_int(s1[0]);
    int signo = recibir_int(s2[0]);
    while (wait(NULL) > 0);
    decir("Resultado: %d - %d = %d", t.a, t.b, signo * dist);
```

```
    return 1;
}

int evaluar_multiplicacion(const Opcion *op, Tarea t)
{
    int b1[2], s1[2], b2[2], s2[2], entre[2];
    (void)op;
    pipes_tarea(b1, s1);
    pipes_tarea(b2, s2);
    pipe(entre);                                    /* contador -> acumulador */
    int fd_cont[] = { b1[0], s1[1], entre[0], entre[1] };
    int fd_acum[] = { b2[0], s2[1], entre[0], entre[1] };
    crear_meeseeks(m_contador, mi_nivel, 1, fd_cont);
    crear_meeseeks(m_acumulador, mi_nivel, 2, fd_acum);
    enviar_tarea(b1[1], t);
    enviar_tarea(b2[1], t);
    int r = recibir_int(s2[0]);
    while (wait(NULL) > 0);
    decir("Resultado: %d * %d = %d", t.a, t.b, r);
    return 1;
}

int evaluar_division(const Opcion *op, Tarea t)
{
    (void)op;
    if (t.b == 0) {
        decir("No se puede dividir entre cero. Lo siento!");
        return 0;
    }
    int b1[2], s1[2], b2[2], s2[2], b3[2], s3[2];
    int div_a_dvd[2], dvd_a_div[2], dvd_a_res[2];
    pipes_tarea(b1, s1);
    pipes_tarea(b2, s2);
    pipes_tarea(b3, s3);
    pipe(div_a_dvd);                                /* divisor  -> dividendo  */
    pipe(dvd_a_div);                                /* dividendo -> divisor   */
    pipe(dvd_a_res);                                /* dividendo -> residuo   */
    int fd_div[] = { b1[0], s1[1], dvd_a_div[0], div_a_dvd[1] };
    int fd_dvd[] = { b2[0], s2[1], div_a_dvd[0], dvd_a_div[1], dvd_a_res[1] };
    int fd_res[] = { b3[0], s3[1], dvd_a_res[0] };
    crear_meeseeks(m_divisor, mi_nivel, 1, fd_div);
    crear_meeseeks(m_dividendo, mi_nivel, 2, fd_dvd);
    crear_meeseeks(m_residuo, mi_nivel, 3, fd_res);
    enviar_tarea(b1[1], t);
    enviar_tarea(b2[1], t);
    enviar_tarea(b3[1], t);
    int cociente = recibir_int(s1[0]);
    int residuo = recibir_int(s3[0]);
    while (wait(NULL) > 0);
    if ((t.a < 0) != (t.b < 0))
        cociente = -cociente;
    decir("Resultado: %d / %d = %d, residuo %d", t.a, t.b, cociente, residuo);
    return 1;
}

/* ========================================================================= */
/*  Logica (dificultad 90): fd[0] = recibe la tarea, fd[1] = resultado     */
/* ========================================================================= */

/* Delegado logico: trabaja un tiempo simulado y calcula la operacion */
void m_logico(int nivel_padre, const int *fd)
{
    int r;
    Tarea t = recibir_tarea(fd[0]);
    presentarse(nivel_padre);
```

```
    int ms = t.duracion_ms > 0 ? t.duracion_ms : 500 + rand() % 4501;
    decir("I am executing request #%d: %s (%.1f s)", t.solicitud, t.texto, ms / 1000.0);
    double t0 = ahora();
    usleep(ms * 1000);
    double espera = ahora() - t0;
    switch (t.operacion) {
    case '&': r = t.a && t.b;   break;
    case '|': r = t.a || t.b;   break;
    case '^': r = !t.a != !t.b; break;
    default:  r = !t.a;         break;   /* '!' */
    }
    enviar_int(fd[1], r);
    enviar_double(fd[1], espera);
}

/* Crea los delegados que pide la dificultad y toma la primera respuesta */
int resolver_logica(const Opcion *op, Tarea t, char operacion)
{
    int baj[2], sub[2];
    int k = ayudantes_necesarios(op->dificultad);
    t.operacion = operacion;
    pipes_tarea(baj, sub);
    int fd[] = { baj[0], sub[1] };
    decir("Dificultad %d: delego en %d Meeseeks", op->dificultad, k);
    for (int j = 0; j < k; j++) {
        crear_meeseeks(m_logico, mi_nivel, j + 1, fd);
        enviar_tarea(baj[1], t);
    }
    int r = recibir_int(sub[0]);
    reportar_espera(recibir_double(sub[0]));
    while (wait(NULL) > 0);
    if (operacion == '!')
        decir("Resultado: NO %d = %d", t.a, r);
    else
        decir("Resultado: %d %s %d = %d", t.a,
              operacion == '&' ? "Y" : operacion == '|' ? "O" : "XOR", t.b, r);
    return 1;
}

int evaluar_y(const Opcion *op, Tarea t)   { return resolver_logica(op, t, '&'); }
int evaluar_o(const Opcion *op, Tarea t)   { return resolver_logica(op, t, '|'); }
int evaluar_xor(const Opcion *op, Tarea t) { return resolver_logica(op, t, '^'); }
int evaluar_no(const Opcion *op, Tarea t)  { return resolver_logica(op, t, '!'); }

/* ========================================================================== */
/*  Programas externos: fork + exec. fd[0] = tarea, fd[1] = codigo salida  */
/* ========================================================================== */

/* Separa el comando en argumentos (por espacios). Devuelve cuantos hay. */
int separar_argumentos(char *comando, char *argv[], int max)
{
    int argc = 0;
    char *parte = strtok(comando, " \t");
    while (parte && argc < max - 1) {
        argv[argc++] = parte;
        parte = strtok(NULL, " \t");
    }
    argv[argc] = NULL;
    return argc;
}

/* Ejecuta el programa con exec en un proceso nuevo y reporta su codigo de salida */
void m_ejecutor(int nivel_padre, const int *fd)
{
    char *argv[16];
```

```
    Tarea t = recibir_tarea(fd[0]);
    presentarse(nivel_padre);
    decir("I am executing request #%d: ejecuto \"%s\"", t.solicitud, t.comando);
    separar_argumentos(t.comando, argv, 16);

    double t0 = ahora();
    pid_t pid = fork();
    if (pid == 0) {
        int nulo = open("/dev/null", O_RDONLY);   /* el programa no lee de la terminal */
        dup2(nulo, STDIN_FILENO);
        signal(SIGPIPE, SIG_DFL);
        execvp(argv[0], argv);
        perror(argv[0]);                          /* solo llega aqui si exec fallo */
        _exit(127);
    }
    int status, codigo;
    waitpid(pid, &status, 0);
    codigo = WIFEXITED(status) ? WEXITSTATUS(status) : 128 + WTERMSIG(status);
    decir("El programa termino con codigo %d en %.3f s", codigo, ahora() - t0);
    enviar_int(fd[1], codigo);
}

/* El evaluador delega la ejecucion en un Meeseeks: completada si sale con 0 */
int ejecutar_programa(Tarea t, const char *comando)
{
    int baj[2], sub[2];
    if (comando)
        snprintf(t.comando, sizeof t.comando, "%s", comando);
    if (t.comando[0] == '\0') {
        decir("No hay ningun comando que ejecutar.");
        return 0;
    }
    pipes_tarea(baj, sub);
    int fd[] = { baj[0], sub[1] };
    crear_meeseeks(m_ejecutor, mi_nivel, 1, fd);
    enviar_tarea(baj[1], t);
    int codigo = recibir_int(sub[0]);
    while (wait(NULL) > 0);
    if (codigo == 0)
        decir("Codigo 0: el programa termino bien.");
    else
        decir("Codigo %d: el programa fallo.", codigo);
    return codigo == 0;
}

int evaluar_fecha(const Opcion *op, Tarea t)   { (void)op; return ejecutar_programa(t,
"date"); }
int evaluar_listar(const Opcion *op, Tarea t)  { (void)op; return ejecutar_programa(t, "ls
-l"); }
int evaluar_falla(const Opcion *op, Tarea t)   { (void)op; return ejecutar_programa(t,
"false"); }
int evaluar_comando(const Opcion *op, Tarea t) { (void)op; return ejecutar_programa(t,
NULL); }

/* ========================================================================== */
/*  Evaluador                                                                 */
/* ========================================================================== */

/* Primer Meeseeks de cada solicitud (N=1, i=1): saluda, evalua y llama a la
 * funcion de la opcion elegida. Sale con 0 si la solicitud se completo. */
void evaluador(const Opcion *op, Tarea t)
{
    setpgid(0, 0);                  /* grupo propio: la Box puede terminarlo todo */
    mi_instancia = 1;
    presentarse(0);                 /* la Box es el nivel 0 */
```

```
    decir("I am evaluating request #%d: \"%s\"", t.solicitud, t.texto);
    int ok = op->funcion(op, t);
    decir("Solicitud #%d %s. Adios!", t.solicitud, ok ? "completada" : "NO completada");
    exit(ok ? 0 : 1);
}

/* ========================================================================== */
/*  Box: configuracion, tiempo maximo, informe y menu                         */
/* ========================================================================== */

static int tiempo_maximo = 300;      /* segundos antes del caos planetario     */
static int duracion_intento_ms = 0;  /* 0 = al azar entre 0.5 y 5 s           */
static int caos_termina = 1;         /* 1 = terminar todo; 0 = avisar y seguir */

/* Lo que la Box recuerda de cada solicitud para el informe */
typedef struct {
    int solicitud;
    char nombre[64];
    double segundos;    /* tiempo total                          */
    double real;        /* tiempo total sin las esperas simuladas */
    int completada;
    int meeseeks;
    int caos;
} Registro;

static Registro registros[MAX_TAREAS];
static int n_registros = 0;

static volatile sig_atomic_t tiempo_agotado = 0;

/* Llega SIGALRM: se cumplio el tiempo maximo de la solicitud */
void al_vencer_tiempo(int sig)
{
    (void)sig;
    tiempo_agotado = 1;
}

/* Espera al evaluador vigilando el tiempo maximo con alarm(). Si se acaba y
 * caos_termina esta activo, termina a todo el grupo de procesos (*caos = 1). */
int esperar_evaluador(pid_t pid, int *caos)
{
    struct sigaction sa;
    int status = 0;

    memset(&sa, 0, sizeof sa);
    sa.sa_handler = al_vencer_tiempo;           /* sin SA_RESTART: waitpid se interrumpe */
    sigaction(SIGALRM, &sa, NULL);
    tiempo_agotado = 0;
    *caos = 0;
    alarm(tiempo_maximo);

    while (waitpid(pid, &status, 0) < 0) {
        if (errno != EINTR || !tiempo_agotado)
            continue;
        tiempo_agotado = 0;
        if (caos_termina) {
            printf("\n*** CAOS PLANETARIO *** Se superaron los %d s. La Box termina a todos
los Meeseeks.\n",
                   tiempo_maximo);
            kill(-pid, SIGKILL);                  /* todo el grupo del evaluador */
            waitpid(pid, &status, 0);
            sem_init(&cont->mutex, 1, 1);         /* alguien pudo morir con el semaforo
tomado */
            *caos = 1;
            break;
```

```
        }
        printf("\n[Box] Se superaron los %d s; sigo esperando a los Meeseeks...\n",
tiempo_maximo);
    }
    alarm(0);
    return status;
}

/* Informe de la Box: todas las solicitudes atendidas */
void informe(void)
{
    int completadas = 0;
    for (int j = 0; j < n_registros; j++)
        completadas += registros[j].completada;
    printf("\n================== INFORME DE LA MEESEEKS BOX ==================\n");
    printf("Solicitudes: %d   Completadas: %d   No completadas: %d\n\n",
           n_registros, completadas, n_registros - completadas);
    printf("%-4s %-36s %9s %9s  %-14s %8s\n", "#", "Solicitud", "T.total", "T.real",
"Estado", "Meeseeks");
    for (int j = 0; j < n_registros; j++) {
        Registro *r = &registros[j];
        char real[16];
        if (r->real < 0)
            snprintf(real, sizeof real, "%9s", "-");
        else
            snprintf(real, sizeof real, "%8.4fs", r->real);
        printf("%-4d %-36.36s %8.4fs %s  %-14s %8d\n", r->solicitud, r->nombre, r-
>segundos,
               real, r->caos ? "caos" : r->completada ? "completada" : "no completada", r-
>meeseeks);
    }
    printf("T.real = tiempo total sin las esperas simuladas de los intentos (- = no hubo
ruta ganadora)\n");
    printf("================================================================\n");
}

/* Submenu de configuracion de la Box */
void configurar(void)
{
    int op, v;
    while (1) {
        printf("\n===== Configuracion =====\n");
        printf("1) Tiempo maximo por solicitud: %d s\n", tiempo_maximo);
        if (duracion_intento_ms > 0)
            printf("2) Duracion de cada intento: %.1f s\n", duracion_intento_ms / 1000.0);
        else
            printf("2) Duracion de cada intento: al azar (0.5 a 5 s)\n");
        printf("3) Al superar el tiempo maximo: %s\n",
               caos_termina ? "caos planetario (terminar todo)" : "solo avisar y seguir
esperando");
        printf("0) Volver\nOpcion: ");
        if (scanf("%d", &op) != 1 || op == 0)
            return;
        if (op == 1) {
            printf("Tiempo maximo en segundos: ");
            if (scanf("%d", &v) == 1 && v > 0)
                tiempo_maximo = v;
        } else if (op == 2) {
            double s;
            printf("Duracion en segundos (0 = al azar): ");
            if (scanf("%lf", &s) == 1 && s >= 0)
                duracion_intento_ms = (int)(s * 1000);
        } else if (op == 3) {
            caos_termina = !caos_termina;
        }
```

```
    }
}

/* Muestra un menu y devuelve la opcion elegida (entra a los submenus).
 * Devuelve NULL si el usuario elige 0 (salir / volver). */
const Opcion *elegir_opcion(const Menu *m, int es_principal)
{
    int id;
    while (1) {
        printf("\n===== %s (pid %d) =====\n", m->titulo, getpid());
        for (int j = 0; j < m->n; j++) {
            const Opcion *o = &m->opciones[j];
            if (o->dificultad >= 0)
                printf("%d) %s (dificultad %d)\n", o->id, o->nombre, o->dificultad);
            else
                printf("%d) %s\n", o->id, o->nombre);
        }
        printf("0) %s\n", es_principal ? "Salir" : "Volver");
        printf("Opcion: ");

        if (scanf("%d", &id) != 1 || id == 0)
            return NULL;
        for (int j = 0; j < m->n; j++) {
            const Opcion *o = &m->opciones[j];
            if (o->id != id)
                continue;
            if (o->submenu == NULL)
                return o;
            const Opcion *elegida = elegir_opcion(o->submenu, 0);
            if (elegida)
                return elegida;
            id = -1;                            /* volvio del submenu */
            break;
        }
        if (id != -1)
            printf("Opcion invalida\n");
    }
}

/* Lee un 0 o un 1 del usuario */
int leer_booleano(const char *pregunta)
{
    int v = 0;
    printf("%s (0 = falso, 1 = verdadero): ", pregunta);
    scanf("%d", &v);
    return v != 0;
}

/* Arma la tarea: copia la opcion y le pide al usuario los datos que falten */
Tarea preparar_tarea(const Opcion *op, int solicitud)
{
    Tarea t;
    memset(&t, 0, sizeof t);
    t.solicitud = solicitud;
    t.dificultad = op->dificultad;
    t.duracion_ms = duracion_intento_ms;
    snprintf(t.texto, sizeof t.texto, "%s", op->nombre);

    switch (op->pide) {
    case PIDE_DOS_NUMEROS:
        printf("Primer numero: ");
        scanf("%d", &t.a);
        printf("Segundo numero: ");
        scanf("%d", &t.b);
        break;
```

```
    case PIDE_DOS_BOOLEANOS:
        t.a = leer_booleano("Primer valor");
        t.b = leer_booleano("Segundo valor");
        break;
    case PIDE_UN_BOOLEANO:
        t.a = leer_booleano("Valor");
        break;
    case PIDE_TEXTO: {
        char d[16];
        printf("Que necesitas? ");
        scanf(" %63[^\n]", t.texto);
        printf("Dificultad 0-100 (100 = muy facil) o 'r' para dejarlo a la suerte: ");
        scanf("%15s", d);
        t.dificultad = (d[0] == 'r' || d[0] == 'R') ? -1 : atoi(d);
        if (t.dificultad > 100)
            t.dificultad = 100;
        break;
    }
    case PIDE_COMANDO:
        printf("Comando (ej. echo hola): ");
        scanf(" %127[^\n]", t.comando);
        snprintf(t.texto, sizeof t.texto, "Ejecutar: %.50s", t.comando);
        break;
    }
    return t;
}

int main(void)
{
    int solicitud = 0;
    pid_t pid;

    setbuf(stdout, NULL);       /* sin buffer: fork no duplica la salida */
    signal(SIGPIPE, SIG_IGN);   /* escribir a un pipe de alguien que ya termino no mata */
    contadores_crear();

    while (1) {
        const Opcion *op = elegir_opcion(&menu_principal, 1);
        if (op == NULL)
            break;
        if (op->pide == PIDE_CONFIG) {
            configurar();
            continue;
        }

        Tarea t = preparar_tarea(op, ++solicitud);

        contadores_reiniciar();
        double t0 = ahora();
        pid = fork();
        if (pid < 0) {
            perror("fork");
            exit(1);
        }
        if (pid == 0)
            evaluador(op, t);                       /* hijo evaluador */
        setpgid(pid, pid);

        int caos;
        int status = esperar_evaluador(pid, &caos);

        if (n_registros < MAX_TAREAS) {
            Registro *r = &registros[n_registros++];
            r->solicitud = t.solicitud;
            snprintf(r->nombre, sizeof r->nombre, "%s", t.texto);
```

```
            r->segundos = ahora() - t0;
            double espera = espera_reportada();
            r->real = (caos || espera < 0) ? -1 : r->segundos - espera;
            r->caos = caos;
            r->completada = !caos && WIFEXITED(status) && WEXITSTATUS(status) == 0;
            r->meeseeks = contadores_total();
        }
        if (caos) {
            printf("El caos planetario obliga a cerrar la Meeseeks Box.\n");
            informe();
            contadores_destruir();
            return 1;
        }
    }

    printf("\nAdios!\n");
    informe();
    contadores_destruir();
    return 0;
}
```